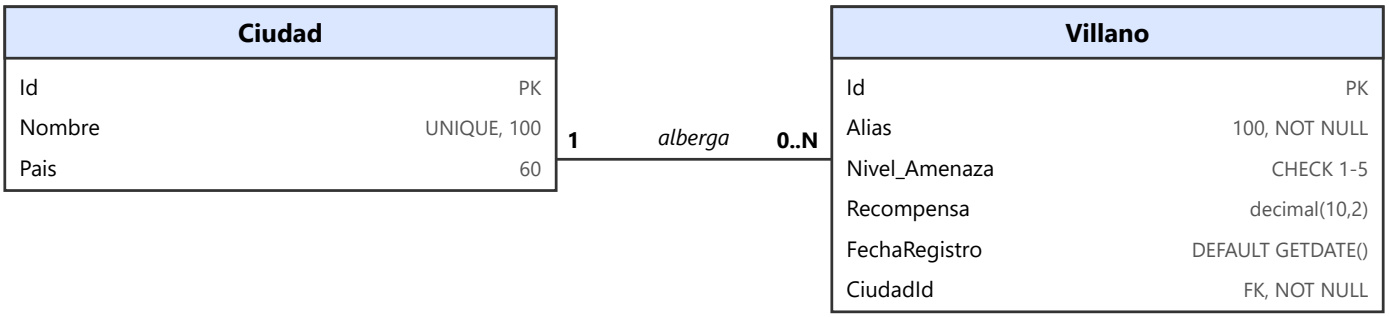
Clase 02 - Práctica de Fluent API con archivos de configuración

Proyecto **HeroesMvc** (ASP.NET Core MVC, .NET 10) · Contexto **HeroesDbContext** · Base **HeroesClase1** (SQL Server) · Sin migraciones: tablas creadas por SQL.

Repositorio: https://github.com/LuisSuarez11/HeroesWeb-RazorPages-DatabaseFirst (carpeta `HeroesMvc`)

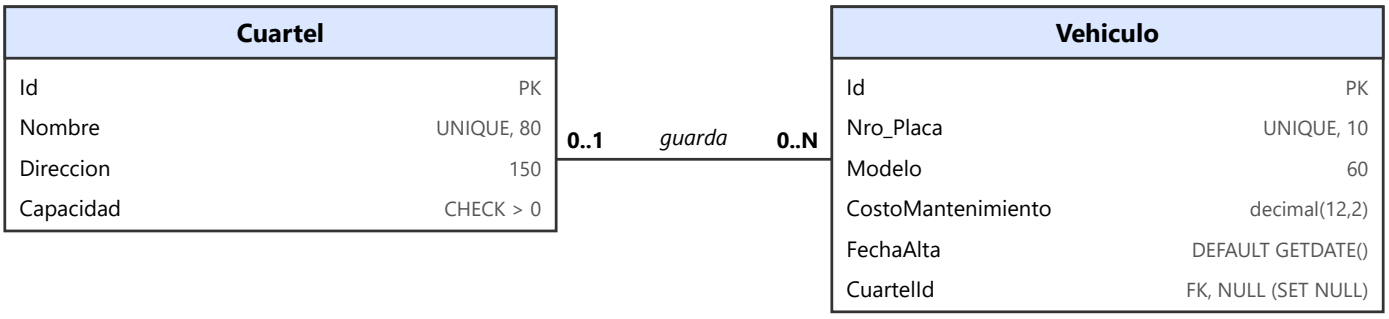
1. DER

DER 1: Ciudad - Villano



Cada villano tiene exactamente una ciudad; una ciudad tiene cero o muchos villanos. La FK va en Villanos (lado muchos).

DER 2 (ejercicio): Cuartel - Vehiculo



2. Entidades (sin atributos)

Models/Ciudad.cs

```
namespace HeroesMvc.Models;

public class Ciudad
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Pais { get; set; } = string.Empty;
    public ICollection<Villano> Villanos { get; set; } = new List<Villano>();
}
```

Models/Villano.cs

```
namespace HeroesMvc.Models;

public class Villano
{
    public int Id { get; set; }
    public string Alias { get; set; } = string.Empty;
    public int NivelAmenaza { get; set; }
    public decimal Recompensa { get; set; }
    public DateTime FechaRegistro { get; set; }
    public int CiudadId { get; set; }
    public Ciudad Ciudad { get; set; } = null!;
}
```

3. Archivos de configuración

Data/Configurations/CiudadConfiguration.cs

```
using HeroesMvc.Models;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace HeroesMvc.Data.Configurations;

public class CiudadConfiguration : IEntityTypeConfiguration<Ciudad>
{
    public void Configure(EntityTypeBuilder<Ciudad> builder)
    {
        builder.ToTable("Ciudades", "dbo");

        builder.HasKey(c => c.Id);

        builder.Property(c => c.Nombre)
            .IsRequired()
            .HasMaxLength(100);

        builder.Property(c => c.Pais)
            .IsRequired()
            .HasMaxLength(60);

        builder.HasIndex(c => c.Nombre)
            .IsUnique()
            .HasDatabaseName("UQ_Ciudades_Nombre");
    }
}
```

Data/Configurations/VillanoConfiguration.cs

```
using HeroesMvc.Models;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace HeroesMvc.Data.Configurations;

public class VillanoConfiguration : IEntityTypeConfiguration<Villano>
{
    public void Configure(EntityTypeBuilder<Villano> builder)
    {
        builder.ToTable("Villanos", "dbo", t =>
            t.HasCheckConstraint(
                "CK_Villanos_NivelAmenaza",
                "[Nivel_Amenaza] BETWEEN 1 AND 5"));

        builder.HasKey(v => v.Id);

        builder.Property(v => v.Alias)
            .IsRequired()
            .HasMaxLength(100);

        builder.Property(v => v.NivelAmenaza)
            .HasColumnName("Nivel_Amenaza");

        builder.Property(v => v.Recompensa)
            .HasPrecision(10, 2);

        builder.Property(v => v.FechaRegistro)
            .HasDefaultValueSql("GETDATE()");

        builder.HasOne(v => v.Ciudad)
            .WithMany(c => c.Villanos)
            .HasForeignKey(v => v.CiudadId)
            .IsRequired()
            .OnDelete(DeleteBehavior.NoAction);
    }
}
```

4. Contexto modificado

Se agregaron los DbSet y las líneas `ApplyConfiguration`; el mapeo de la clase 2 se conserva.

Data/HeroesDbContext.cs

```
using HeroesMvc.Data.Configurations;
using HeroesMvc.Models;
using Microsoft.EntityFrameworkCore;

namespace HeroesMvc.Data;

public class HeroesDbContext : DbContext
{
    public HeroesDbContext(DbContextOptions<HeroesDbContext> options)
        : base(options)
    {
    }

    // Clase 2
    public DbSet<Equipo> Equipos => Set<Equipo>();
    public DbSet<Heroe> Heroes => Set<Heroe>();
    public DbSet<Ficha> Fichas => Set<Ficha>();
    public DbSet<Superpoder> Superpoderes => Set<Superpoder>();
    public DbSet<HeroeSuperpoder> HeroeSuperpoderes => Set<HeroeSuperpoder>();

    // Clase 02 (ejemplo)
    public DbSet<Ciudad> Ciudades => Set<Ciudad>();
    public DbSet<Villano> Villanos => Set<Villano>();

    // Clase 02 (ejercicio)
    public DbSet<Cuartel> Cuarteles => Set<Cuartel>();
    public DbSet<Vehiculo> Vehiculos => Set<Vehiculo>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // ---- Mapeo de la clase 2 (se conserva en el contexto) ----
        modelBuilder.Entity<Equipo>(e =>
        {
            e.ToTable("Equipos", "dbo");
            e.HasKey(x => x.Id);
            e.Property(x => x.Nombre).IsRequired().HasMaxLength(100);
            e.HasIndex(x => x.Nombre).IsUnique().HasDatabaseName("UQ_Equipos_Nombre");
        });

        modelBuilder.Entity<Heroe>(e =>
        {
            e.ToTable("Heroes", "dbo");
            e.HasKey(x => x.Id);
            e.Property(x => x.Nombre).IsRequired().HasMaxLength(100);
            e.Property(x => x.Ciudad).IsRequired().HasMaxLength(100);
            e.HasOne(x => x.Equipo)
                .WithMany(q => q.Heroes)
                .HasForeignKey(x => x.EquipoId)
                .IsRequired(false)
                .OnDelete(DeleteBehavior.SetNull);
        });

        modelBuilder.Entity<Ficha>(e =>
        {
            e.ToTable("Fichas", "dbo");
```

```
            e.HasKey(x => x.HeroeId);
            e.Property(x => x.IdentidadSecreta).HasMaxLength(100);
            e.Property(x => x.Debilidad).HasMaxLength(150);
            e.HasOne(x => x.Heroe)
                .WithOne(h => h.Ficha)
                .HasForeignKey<Ficha>(x => x.HeroeId)
                .OnDelete(DeleteBehavior.Cascade);
        });

        modelBuilder.Entity<Superpoder>(e =>
        {
            e.ToTable("Superpoderes", "dbo");
            e.HasKey(x => x.Id);
            e.Property(x => x.Nombre).IsRequired().HasMaxLength(100);
            e.Property(x => x.Descripcion).HasMaxLength(250);
        });

        modelBuilder.Entity<HeroeSuperpoder>(e =>
        {
            e.ToTable("HeroeSuperpoder", "dbo");
            e.HasKey(x => new { x.HeroeId, x.SuperpoderId });
            e.HasOne(x => x.Heroe)
                .WithMany(h => h.HeroeSuperpoderes)
                .HasForeignKey(x => x.HeroeId)
                .OnDelete(DeleteBehavior.Cascade);
            e.HasOne(x => x.Superpoder)
                .WithMany(s => s.HeroeSuperpoderes)
                .HasForeignKey(x => x.SuperpoderId)
                .OnDelete(DeleteBehavior.Cascade);
        });

        // ---- Clase 02: archivos de configuracion ----
        modelBuilder.ApplyConfiguration(new CiudadConfiguration());
        modelBuilder.ApplyConfiguration(new VillanoConfiguration());

        // ---- Ejercicio Cuartel-Vehiculo ----
        modelBuilder.ApplyConfiguration(new CuartelConfiguration());
        modelBuilder.ApplyConfiguration(new VehiculoConfiguration());
    }
}
```

5. Controlador de pruebas

Controllers/LaboratorioEfController.cs

```
using HeroesMvc.Data;
using HeroesMvc.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace HeroesMvc.Controllers;

public class LaboratorioEfController : Controller
{
    private readonly HeroesDbContext _context;

    public LaboratorioEfController(HeroesDbContext context)
    {
        _context = context;
    }

    [HttpGet]
    public async Task<IActionResult> Relaciones(int id)
    {
        var heroe = await _context.Heroes
            .Include(h => h.Equipo)
            .Include(h => h.Ficha)
            .Include(h => h.HeroeSuperpoderes).ThenInclude(hs => hs.Superpoder)
            .FirstOrDefaultAsync(h => h.Id == id);

        if (heroe is null)
            return NotFound();

        return Json(new
        {
            heroe.Id,
            heroe.Nombre,
            heroe.Ciudad,
            Equipo = heroe.Equipo?.Nombre,
            IdentidadSecreta = heroe.Ficha?.IdentidadSecreta,
            Debilidad = heroe.Ficha?.Debilidad,
            Superpoderes = heroe.HeroeSuperpoderes.Select(hs => hs.Superpoder.Nombre).ToList()
        });
    }

    [HttpGet]
    public async Task<IActionResult> Ciudad(int id)
    {
        var ciudad = await _context.Ciudades
            .Include(c => c.Villanos)
            .FirstOrDefaultAsync(c => c.Id == id);

        if (ciudad is null)
            return NotFound();

        return Json(new
        {
            ciudad.Id,
            ciudad.Nombre,
            ciudad.Pais,
            Villanos = ciudad.Villanos.Select(v => new
            {
                v.Alias,
                v.NivelAmenaza,
```

```
                v.Recompensa,
                v.FechaRegistro
            }).ToList()
        });
    }

    [HttpGet]
    public IActionResult ModeloVillano()
    {
        var entidad = _context.Model.FindEntityType(typeof(Villano))!;

        return Json(new
        {
            Tabla = entidad.GetTableName(),
            Esquema = entidad.GetSchema(),
            Columnas = entidad.GetProperties().Select(p => new
            {
                Propiedad = p.Name,
                Columna = p.GetColumnName(),
                Obligatoria = !p.IsNullable,
                Longitud = p.GetMaxLength(),
                Precision = p.GetPrecision(),
                Escala = p.GetScale(),
                DefaultSql = p.GetDefaultValueSql()
            })
        });
    }

    [HttpGet]
    public async Task<IActionResult> Cuartel(int id)
    {
        var cuartel = await _context.Cuarteles
            .Include(c => c.Vehiculos)
            .FirstOrDefaultAsync(c => c.Id == id);

        if (cuartel is null)
            return NotFound();

        return Json(new
        {
            cuartel.Id,
            cuartel.Nombre,
            cuartel.Capacidad,
            Vehiculos = cuartel.Vehiculos.Select(v => new
            {
                v.Placa,
                v.Modelo,
                v.CostoMantenimiento,
                v.FechaAlta
            }).ToList()
        });
    }
}
```

6. Pruebas A a D (capturas)

A. Consulta con relación

Impresión con sangría


```
{"id":1,"nombre":"La Paz","pais":"Bolivia","villanos":[{"alias":"Sombra
Roja","nivelAmenaza":4,"recompensa":15000.50,"fechaRegistro":"2026-10-11T16:16:35.83"},{"alias":"Doctor
Niebla","nivelAmenaza":3,"recompensa":8200.00,"fechaRegistro":"2026-10-11T16:16:35.83"}]}
```


/LaboratorioEf/Ciudad/1 (La Paz): Sombra Roja y Doctor Niebla con nivel, recompensa y fecha de SQL Server

Impresión con sangría


```
{"id":2,"nombre":"Santa Cruz","pais":"Bolivia","villanos":[]}
```


/LaboratorioEf/Ciudad/2 (Santa Cruz): lista de villanos vacía

B. Metadatos del modelo

Impresión con sangría


```
{"tabla":"Villanos","esquema":"dbo","columnas":
[{"propiedad":"Id","columna":"Id","obligatoria":true,"longitud":null,"precision":null,"escala":null,"defau
ltSql":null},
{"propiedad":"Alias","columna":"Alias","obligatoria":true,"longitud":100,"precision":null,"escala":null,"d
efaultSql":null},
{"propiedad":"CiudadId","columna":"CiudadId","obligatoria":true,"longitud":null,"precision":null,"escala":
null,"defaultSql":null},
{"propiedad":"FechaRegistro","columna":"FechaRegistro","obligatoria":true,"longitud":null,"precision":null
,"escala":null,"defaultSql":"GETDATE()"},
{"propiedad":"NivelAmenaza","columna":"Nivel_Amenaza","obligatoria":true,"longitud":null,"precision":null,
"escala":null,"defaultSql":null},
{"propiedad":"Recompensa","columna":"Recompensa","obligatoria":true,"longitud":null,"precision":10,"escala
":2,"defaultSql":null}]}
```

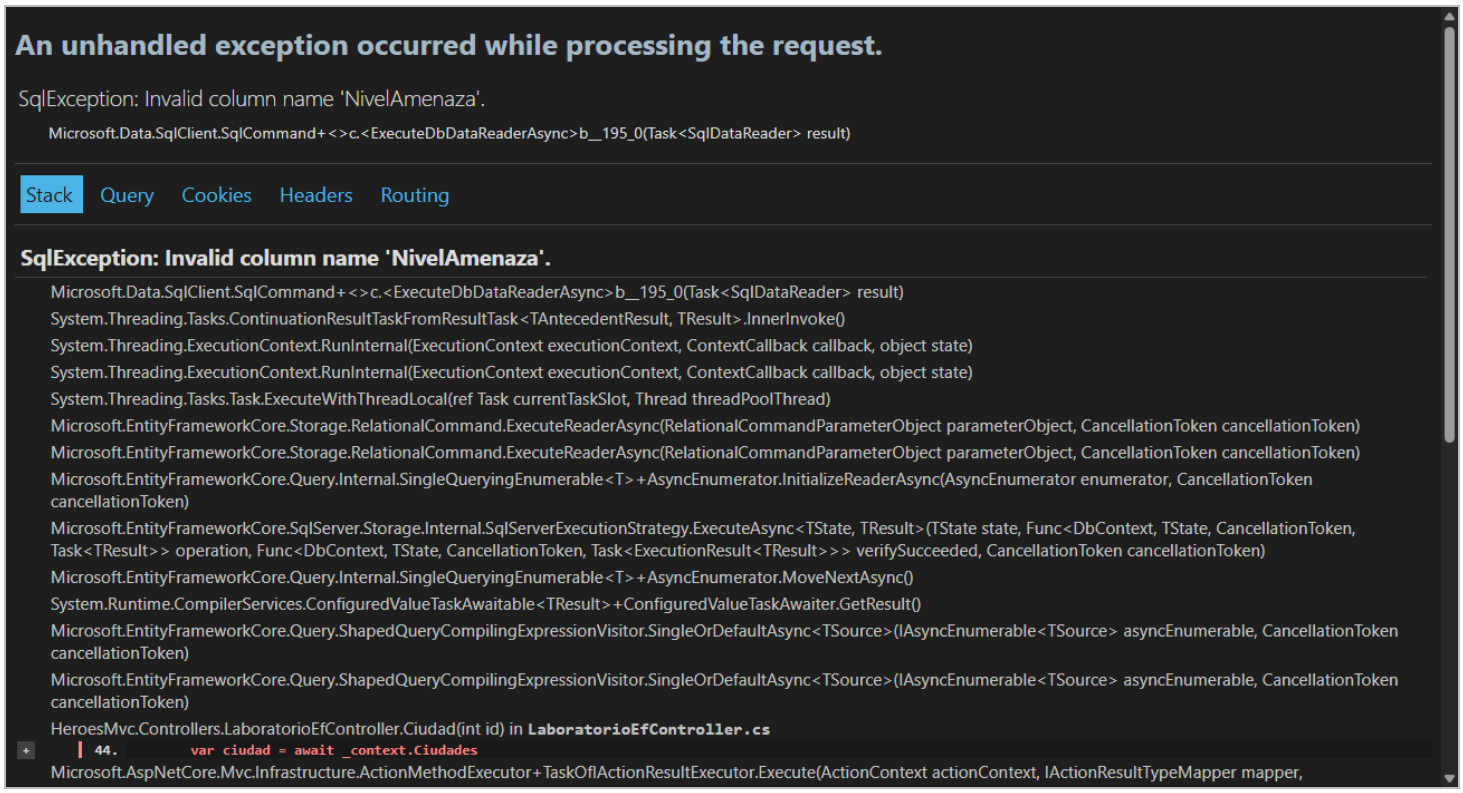

NivelAmenaza -> Nivel_Amenaza; Alias longitud 100; Recompensa precisión 10 y escala 2; FechaRegistro con GETDATE()

C. Prueba de quitar la configuración (VillanoConfiguration comentada)

Impresión con sangría


```
{"tabla":"Villanos","esquema":null,"columnas":
[{"propiedad":"Id","columna":"Id","obligatoria":true,"longitud":null,"precision":null,"escala":null,"defau
ltSql":null},
{"propiedad":"Alias","columna":"Alias","obligatoria":true,"longitud":null,"precision":null,"escala":null,"
defaultSql":null},
{"propiedad":"CiudadId","columna":"CiudadId","obligatoria":true,"longitud":null,"precision":null,"escala":
null,"defaultSql":null},
{"propiedad":"FechaRegistro","columna":"FechaRegistro","obligatoria":true,"longitud":null,"precision":null
,"escala":null,"defaultSql":null},
{"propiedad":"NivelAmenaza","columna":"NivelAmenaza","obligatoria":true,"longitud":null,"precision":null,"
escala":null,"defaultSql":null},
{"propiedad":"Recompensa","columna":"Recompensa","obligatoria":true,"longitud":null,"precision":null,"esca
la":null,"defaultSql":null}]}
```


ModeloVillano: la columna pasa a NivelAmenaza y desaparecen longitud, precisión y default



Ciudad/1 falla con: Invalid column name 'NivelAmenaza'. Al quitar el comentario todo vuelve a funcionar.

D. Restricciones en SSMS (todas con ROLLBACK, sin cambiar datos)

```
Changed database context to 'HeroesClase1'.
=== D1: nombre de ciudad repetido (UQ_Ciudades_Nombre) ===

(0 rows affected)
Resultado
---------
Violation of UNIQUE KEY constraint 'UQ_Ciudades_Nombre'. Cannot insert duplicate key in object 'dbo.Ciudades'. The duplicate key value is (La Paz).

(1 rows affected)
=== D2: nivel de amenaza fuera de rango (CK_Villanos_NivelAmenaza) ===

(0 rows affected)
Resultado
---------
The UPDATE statement conflicted with the CHECK constraint "CK_Villanos_NivelAmenaza". The conflict occurred in database "HeroesClase1", table "dbo.Villanos", column 'Nivel_Amenaza'.

(1 rows affected)
=== D3: borrar ciudad con villanos (FK_Villanos_Ciudades) ===

(0 rows affected)
Resultado
---------
The DELETE statement conflicted with the REFERENCE constraint "FK_Villanos_Ciudades". The conflict occurred in database "HeroesClase1", table "dbo.Villanos", column 'CiudadId'.

(1 rows affected)
```

UQ_Ciudades_Nombre, CK_Villanos_NivelAmenaza y FK_Villanos_Ciudades devuelven error

7. Respuestas a las preguntas de comprensión

Preguntas de comprensión

1. ¿Qué ventaja tiene poner la configuración en clases `IEntityTypeConfiguration` en vez de escribirla en `OnModelCreating`?

Cada entidad tiene su propio archivo, fácil de encontrar y revisar, y `OnModelCreating` se mantiene corto en vez de crecer sin control. Además, el modelo se puede probar y modificar por partes (por ejemplo, comentar una sola línea de registro).

2. ¿Qué diferencia hay entre `ApplyConfiguration` y `ApplyConfigurationsFromAssembly`?

`ApplyConfiguration(new X())` registra una configuración concreta, de forma explícita: se ve qué se aplica y es fácil desactivar una. `ApplyConfigurationsFromAssembly(assembly)` busca en el ensamblado todas las clases que implementan `IEntityTypeConfiguration<T>` y las aplica de una vez; es más cómodo, pero menos visible y no permite omitir una con solo comentar una línea.

3. ¿Por qué falló la consulta al comentar `VillanoConfiguration`?

Sin la configuración, EF aplica las convenciones y busca la columna con el nombre de la propiedad (`NivelAmenaza`), pero en la base se llama `Nivel_Amenaza`. Por eso SQL Server responde `Invalid column name 'NivelAmenaza'`. El archivo de configuración es lo que hace coincidir el modelo con la base. (La Fluent API tiene la mayor prioridad: sobrescribe a las convenciones y a las anotaciones, así que sin ella el modelo vuelve a las convenciones.)

4. Si `HasCheckConstraint` no crea la restricción en esta práctica, ¿qué la impone?

La impone SQL Server: la restricción `CK_Villanos_NivelAmenaza` ya existe en la tabla porque se creó con el script SQL. En el modelo, `HasCheckConstraint` solo la describe para que el modelo quede igual a la base; no hay migración que la genere.

5. ¿Por qué la relación se configura solo en `VillanoConfiguration`?

Porque la relación se configura una sola vez, desde el dependiente (el lado que tiene la FK `CiudadId`). Configurarla también desde `CiudadConfiguration` duplicaría el mapeo y podría crear una FK extra (por ejemplo `CiudadId1`).

6. ¿Qué pasa al intentar borrar una ciudad con villanos, y por qué?

El borrado falla con un conflicto de la restricción `FK_Villanos_Ciudades`. En el script la FK es `ON DELETE NO ACTION` y en el modelo `OnDelete(DeleteBehavior.NoAction)`: no se puede borrar una ciudad mientras haya villanos que la referencian (prueba D3: error y sin cambios).

8. Ejercicio: Cuarteles y Vehículos

Models/Cuartel.cs

```
namespace HeroesMvc.Models;

public class Cuartel
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Direccion { get; set; } = string.Empty;
    public int Capacidad { get; set; }
    public ICollection<Vehiculo> Vehiculos { get; set; } = new List<Vehiculo>();
}
```

Models/Vehiculo.cs

```
namespace HeroesMvc.Models;

public class Vehiculo
{
    public int Id { get; set; }
    public string Placa { get; set; } = string.Empty;
    public string Modelo { get; set; } = string.Empty;
    public decimal CostoMantenimiento { get; set; }
    public DateTime FechaAlta { get; set; }
    public int? CuartelId { get; set; }
    public Cuartel? Cuartel { get; set; }
}
```

Data/Configurations/CuartelConfiguration.cs

```
using HeroesMvc.Models;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace HeroesMvc.Data.Configurations;

public class CuartelConfiguration : IEntityTypeConfiguration<Cuartel>
{
    public void Configure(EntityTypeBuilder<Cuartel> builder)
    {
        builder.ToTable("Cuarteles", "dbo", t =>
            t.HasCheckConstraint(
                "CK_Cuarteles_Capacidad", "[Capacidad] > 0"));

        builder.HasKey(c => c.Id);

        builder.Property(c => c.Nombre)
            .IsRequired()
            .HasMaxLength(80);

        builder.Property(c => c.Direccion)
            .IsRequired()
            .HasMaxLength(150);

        builder.HasIndex(c => c.Nombre)
            .IsUnique()
            .HasDatabaseName("UQ_Cuarteles_Nombre");
    }
}
```

Data/Configurations/VehiculoConfiguration.cs

```
using HeroesMvc.Models;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace HeroesMvc.Data.Configurations;

public class VehiculoConfiguration : IEntityTypeConfiguration<Vehiculo>
{
    public void Configure(EntityTypeBuilder<Vehiculo> builder)
    {
        builder.ToTable("Vehiculos", "dbo");

        builder.HasKey(v => v.Id);

        builder.Property(v => v.Placa)
            .IsRequired()
            .HasMaxLength(10)
            .HasColumnName("Nro_Placa");

        builder.HasIndex(v => v.Placa)
            .IsUnique()
            .HasDatabaseName("UQ_Vehiculos_Placa");

        builder.Property(v => v.Modelo)
            .IsRequired()
            .HasMaxLength(60);

        builder.Property(v => v.CostoMantenimiento)
            .HasPrecision(12, 2);

        builder.Property(v => v.FechaAlta)
            .HasDefaultValueSql("GETDATE()");

        builder.HasOne(v => v.Cuartel)
            .WithMany(c => c.Vehiculos)
            .HasForeignKey(v => v.CuartelId)
            .IsRequired(false)
            .OnDelete(DeleteBehavior.SetNull);
    }
}
```

Pruebas del ejercicio

Impresión con sangría

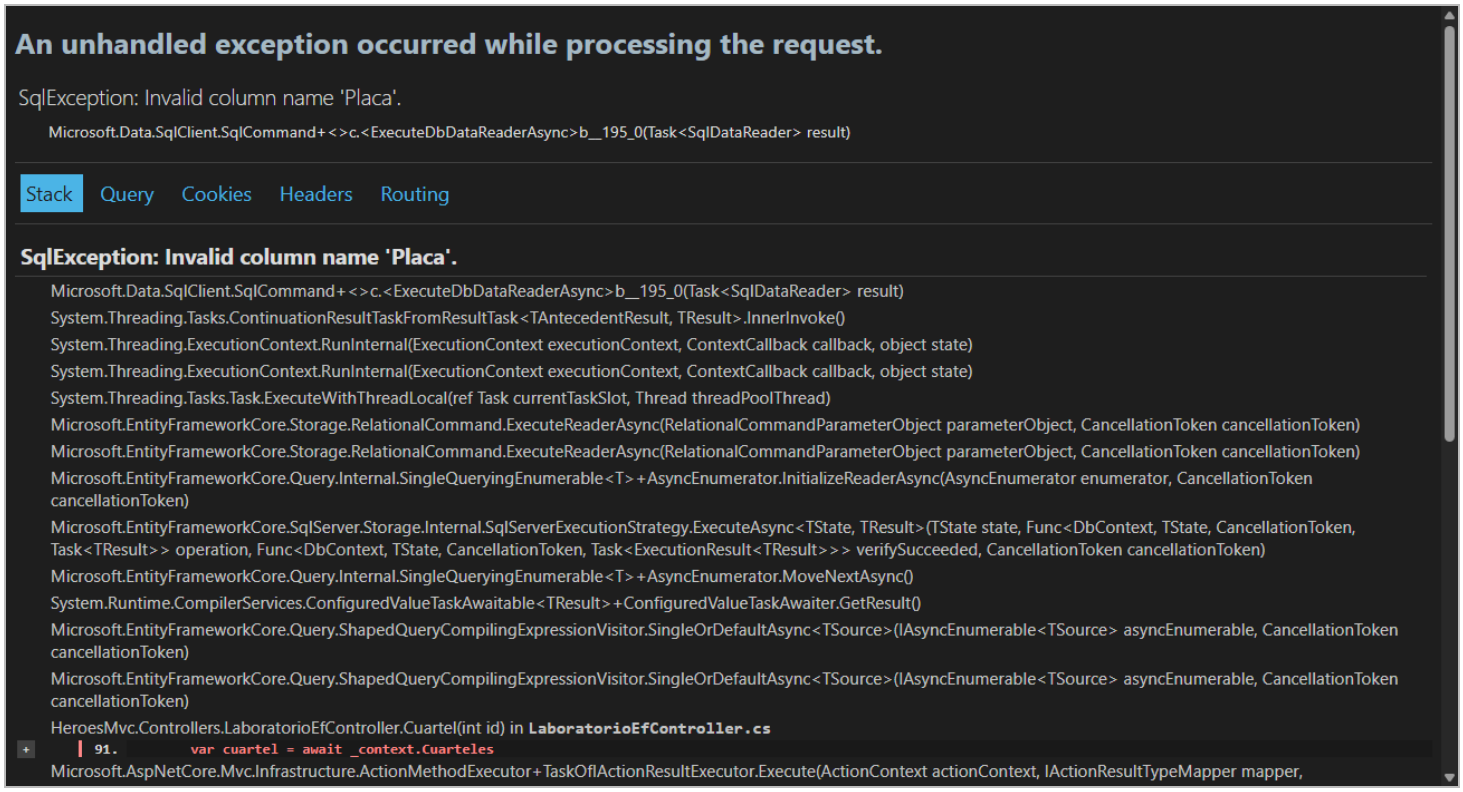

```
{"id":1,"nombre":"Base Illimani","capacidad":20,"vehiculos":[{"placa":"2345-ABC","modelo":"Halc\u00F3n
X1","costoMantenimiento":8500.00,"fechaAlta":"2026-10-11T16:16:35.83"},{"placa":"9876-XYZ","modelo":"Rayo
T2","costoMantenimiento":4200.75,"fechaAlta":"2026-10-11T16:16:35.83"}]}
```


Cuartel/1 (Base Illimani): Halcón X1 y Rayo T2 con fecha de alta

Impresión con sangría


```
{"id":2,"nombre":"Base Oriente","capacidad":12,"vehiculos":[]}
```


Cuartel/2 (Base Oriente): lista de vehículos vacía



Con VehiculoConfiguration comentada: Invalid column name 'Placa'

```
=== E4: placa repetida 2345-ABC (UQ_Vehiculos_Placa) ===

(0 rows affected)
Resultado
---------
Violation of UNIQUE KEY constraint 'UQ_Vehiculos_Placa'. Cannot insert duplicate key in object 'dbo.Vehiculos'. The duplicate key value is (2345-ABC).

(1 rows affected)
=== E5: capacidad 0 (CK_Cuarteles_Capacidad) ===

(0 rows affected)
Resultado
---------
The UPDATE statement conflicted with the CHECK constraint "CK_Cuarteles_Capacidad". The conflict occurred in database "HeroesClase1", table "dbo.Cuarteles", column 'Capacidad'.

(1 rows affected)
=== E6: borrar Base Illimani (SET NULL), luego ROLLBACK ===

(1 rows affected)
Nro_Placa Modelo CuartelId
--------- ------ ---------
2345-ABC Halcón X1 NULL
9876-XYZ Rayo T2 NULL
1111-DEF Tortuga M3 NULL

(3 rows affected)
=== Verificacion: los datos siguen intactos ===
Ciudades Villanos Cuarteles VehiculosConCuartel
-------- -------- --------- --------------------
2 2 2 2

(1 rows affected)
```

Placa repetida (UQ_Vehiculos_Placa), capacidad 0 (CK_Cuarteles_Capacidad) y borrado de Base Illimani con vehículos en NULL

Respuestas del ejercicio

Preguntas del ejercicio Cuartel – Vehículo

1. ¿Por qué `Vehiculo.CuartelId` es `int?` y `Villano.CiudadId` es `int`?

Un vehículo puede no tener cuartel (columna `NULL`); un villano siempre tiene ciudad (`NOT NULL`). La nulabilidad de la FK en C# debe coincidir con `NULL` / `NOT NULL` en SQL. Si fuera `int`, EF la trataría como obligatoria y no coincidiría con la columna.

2. ¿Qué diferencia hay entre `SetNull` y `NoAction` al borrar el principal?

`SetNull` permite borrar el cuartel y deja en `NULL` la FK de sus vehículos (prueba E6: los tres vehículos quedaron con `CuartelId = NULL`). `NoAction` impide el borrado mientras queden filas que lo referencian (como Ciudad–Villano).

3. ¿Por qué el CHECK de capacidad va en `CuartelConfiguration` y no en `VehiculoConfiguration`?

Porque la regla habla de una columna de la tabla `Cuarteles` (`Capacidad`). Cada archivo de configuración describe solo su propia tabla.

4. ¿Por qué la prueba de comentar la configuración falla con `Placa` y no con `Modelo`?

Sin la configuración, EF busca la columna por el nombre de la propiedad. `Modelo` se llama igual en C# y en SQL, así que se encuentra; `Placa` no, porque en la base la columna es `Nro_Placa`, y eso solo lo resuelve `HasColumnName("Nro_Placa")`.